

Road Network Planning with Traffic Sign Recognition

Road Network Planning with Traffic Sign Recognition

1. Course Content
2. Preparation Work
 - 2.1 Start `kilted_agent` Agent
 - 2.2 Prepare Site Layout
3. Run Case Program
 - 3.1 Start Road Network Navigation
 - 3.2 Start Autonomous Driving
 - 3.3 Case Demonstration
 - 3.4 Other Traffic Signs
4. Source Code Analysis

1. Course Content

[!NOTE]

- This course is an extended tutorial for autonomous driving track users, requiring the use of the accompanying track map and traffic signal signs to proceed with the course case study
- This course requires completion of the [07.Road Network Planning and Navigation—4.Inter-Road Network Planning and Navigation] chapter content first
- Combine traffic signal recognition functionality in the track to achieve response to traffic light signals and traffic sign signals on the basis of road network planning autonomous driving

2. Preparation Work

2.1 Start `kilted_agent` Agent

- Use the following command to start the microROS agent and connect to the chassis

[!WARNING]

If you use the `start_agent` command to start the microROS agent, you need to close the previous agent first. Note that this is different from the previous microROS agent startup command

```
start_kilted_agent
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 80x24
subscriber created | client_key: 0x14F22590, subscriber_id: 0x000(4), partici
pant_id: 0x000(1)
[1763535321.445792] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1763535321.449556] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1763535321.452876] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1763535321.457712] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1763535321.461959] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1763535321.466141] info | ProxyClient.cpp | create_subscriber | s
subscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1763535321.473112] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
```

- Start ros2kilted container (if already started, no need to repeat)

```
bringup_ros2kilted
```

```
jetson@yahboom: ~
jetson@yahboom: ~ 102x39
jetson@yahboom:~$ bringup_ros2kilted
access control disabled, clients can connect from any host
jetson@yahboom:~$
: Container ros2_kilted-ros2_kilted-1 Starting
1.4s
```

[!WARNING]

For Raspberry Pi 5 users, if the ROS container is not started, you need to start the container first,

```
bringup_m3
```

Open a container terminal,

```
exec_m3
```

Open ~/.bashrc, uncomment this line, and change it to the DDS communication middleware as shown in the figure (only the road network planning navigation chapter needs to use cyclondds, **please restore the environment variables after completing the road network chapter!!**)

```
vim ~/.bashrc
```

- RDK & PI5 mainboard

```
131 #ROSMaster-M3 Code space environment variables
132 export PYTHONPATH="/home/sunrise/yahboomM3_ws/.venv/lib/python3.10/site-packages:$PYTHONPATH"
133 export ROS_DOMAIN_ID=16
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 #unset RMW_IMPLEMENTATION
136 export ROBOT_TYPE="ROSMaster-M3"
137 export MULTI_ROBOT="" #robot1 or robot2
138 #=====雷达和相机型号=====
139 #=====Radar and camera models=====
140 #=====
141 #单雷达: single          双雷达: dual
142 #Single radar: single    Dual radar: dual
143 export LASER_TYPE=single
144 #export LASER_TYPE=dual
145 "
```

- ORIN mainboard

```
133 export ROS_DOMAIN_ID=30
134 export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
135 export ROBOT_TYPE="ROSMaster-M3"
136 export MULTI_ROBOT=""
137 "
```

```
:wq #Save and exit
source ~/.bashrc #Update environment
```

2.2 Prepare Site Layout

- Refer to [08. Autonomous Driving expand(Need purchase track map)]—[01.Environment Setup & Precautions] for site preparation. The placement of traffic signs is not fixed and should be arranged according to your own needs



3. Run Case Program

[!IMPORTANT]

- Need to build the track map's grid map: map, pose map: pose_map, and road network description file: roadnet_file in advance according to [08.Autonomous Driving expand(Need purchase track map)]-[13.Road Network Planning with Track Map]

3.1 Start Road Network Navigation

- Open a terminal on the host machine to start the vehicle's radar fusion node and odometer fusion

```
ros2 launch m3_bringup car_base.launch.py
```

- Start ros2_kilted container

```
bringup_ros2kilted
```

- Enter ros2kilted container terminal

```
exec_ros2kilted
```

- (RDK mainboard) Visualization is started on the accompanying virtual machine `Rosmaster-RoadNet-Ubuntu24.04`,

```
rviz2 -d /home/yahboom/code_ws/src/road_net_route/rviz/nav2_view.rviz
```

- Start road network planning navigation node in container terminal

```
ros2 launch road_net_route roadnet_nav2.launch.py
```

[!TIP]

Parameter Description

map Grid map file name to be loaded, default value is `map.yaml`

pose_map Pose map file to be loaded. When navigation global positioning uses slam_toolbox pure positioning, the pose map must be passed in, default value is `map`

roadnet_file Road network description file, default value is `map.geojson`

3.2 Start Autonomous Driving

- Run in host machine terminal

```
ros2 launch m3_bringup auto_drive.launch.py combine_nav:=True unkeeplane:=True
```

[!NOTE]

Parameter Description

- **combine_nav** Whether to combine road network navigation for autonomous driving, default is False. If True, the motion control of autonomous driving is provided by road network planning navigation, and traffic light response will directly replace the interruption and recovery acting on road network planning navigation
- **unkeeplane** Whether to close lane line detection model and thread, default is False. If road network navigation is started for autonomous driving, lane line detection and lane keeping functions will be automatically closed

3.3 Case Demonstration

- Here we demonstrate autonomous driving from road network point 10 to road network point 1 as an example; we need to first query the coordinates near point 1, first use regular navigation to let the vehicle navigate to the target point, or directly move the vehicle to the target position, then use the `2D Pose Estimate` tool for relocalization
- After the vehicle reaches the target position that needs to be recorded, use the following command to get the current robot's coordinates in the map

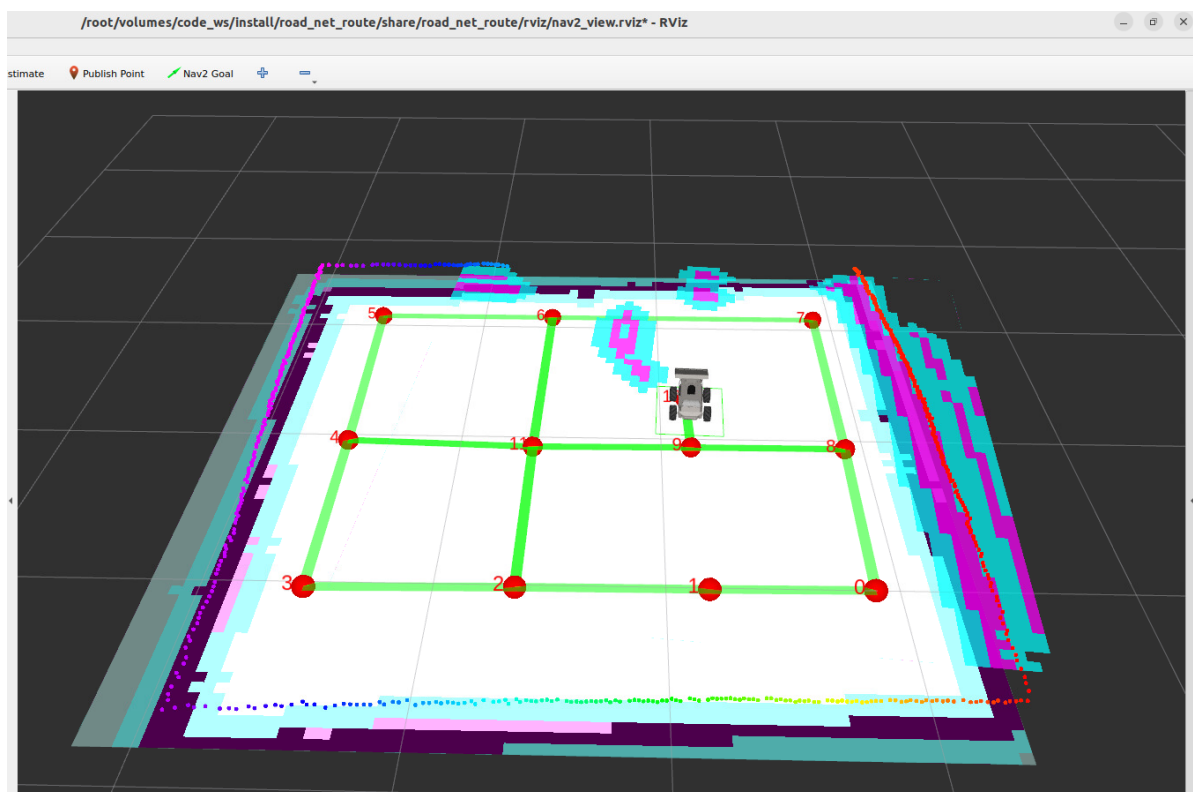
```
ros2 run tf2_ros tf2_echo map base_footprint
```



```
ahboom: ~
jetson@yahboom: ~ 70x19
- Rotation: in RPY (radian) [0.000, 0.000, -0.104]
- Rotation: in RPY (degree) [0.000, 0.000, -5.946]
- Matrix:
  0.995  0.104  0.000  2.383
 -0.104  0.995 -0.000 -1.352
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
At time 1763608206.239000000
- Translation: [2.383, -1.352, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.052, 0.999]
- Rotation: in RPY (radian) [0.000, 0.000, -0.104]
- Rotation: in RPY (degree) [0.000, 0.000, -5.959]
- Matrix:
  0.995  0.104  0.000  2.383
 -0.104  0.995 -0.000 -1.352
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
^C[INFO] [1763608207.095851031] [rclcpp]: signal_handler(signum=2)
jetson@yahboom:~$
```

- Let the vehicle return to road network point 10, publish endpoint information in terminal (fill in actual target pose)

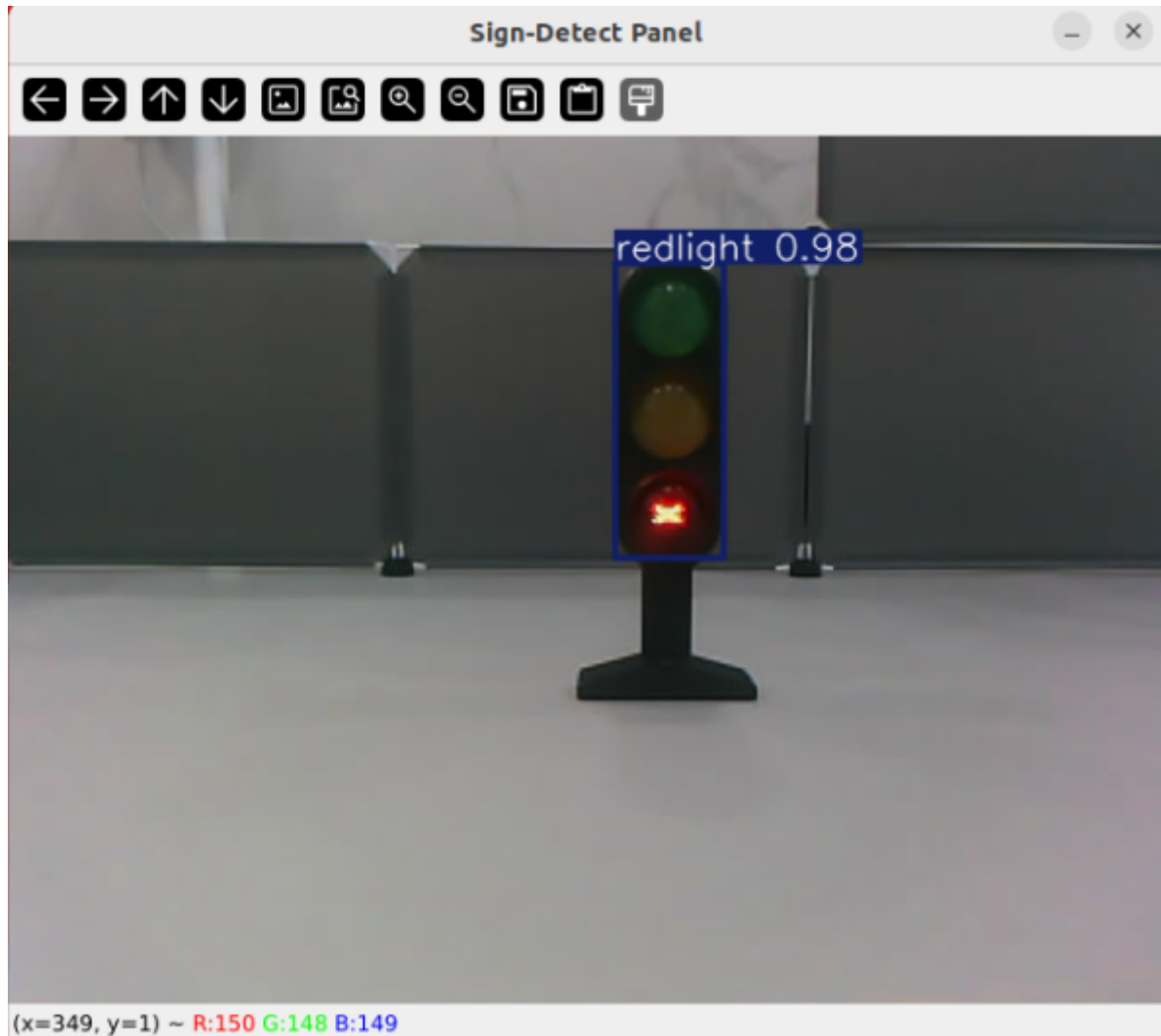
```
ros2 topic pub -1 /road_net_nav geometry_msgs/PoseStamped "{
  pose: {
    position: {x: xx, y: xx, z: 0.0},
    orientation: {x: 0.0, y: 0.0, z: xx, w: xx}
  }
}"
```

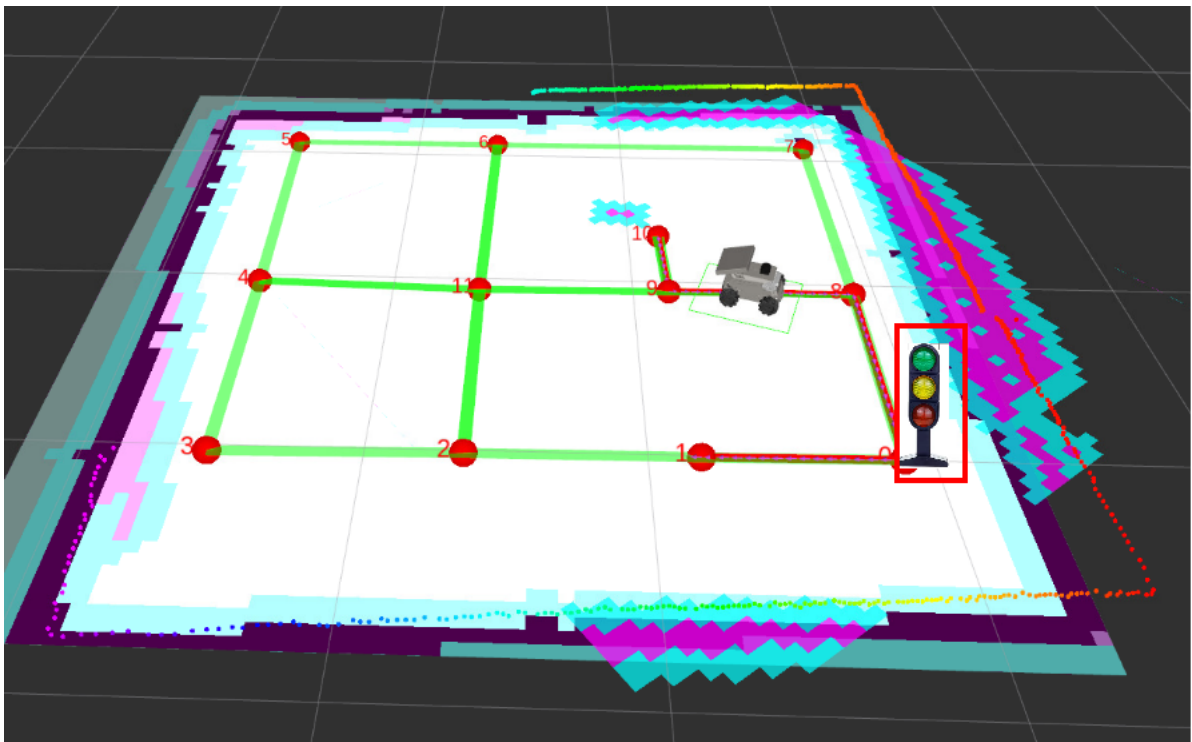


- Place traffic light props next to the path in front of the lane and turn on red light mode. When the vehicle detects a red light, it will stop immediately

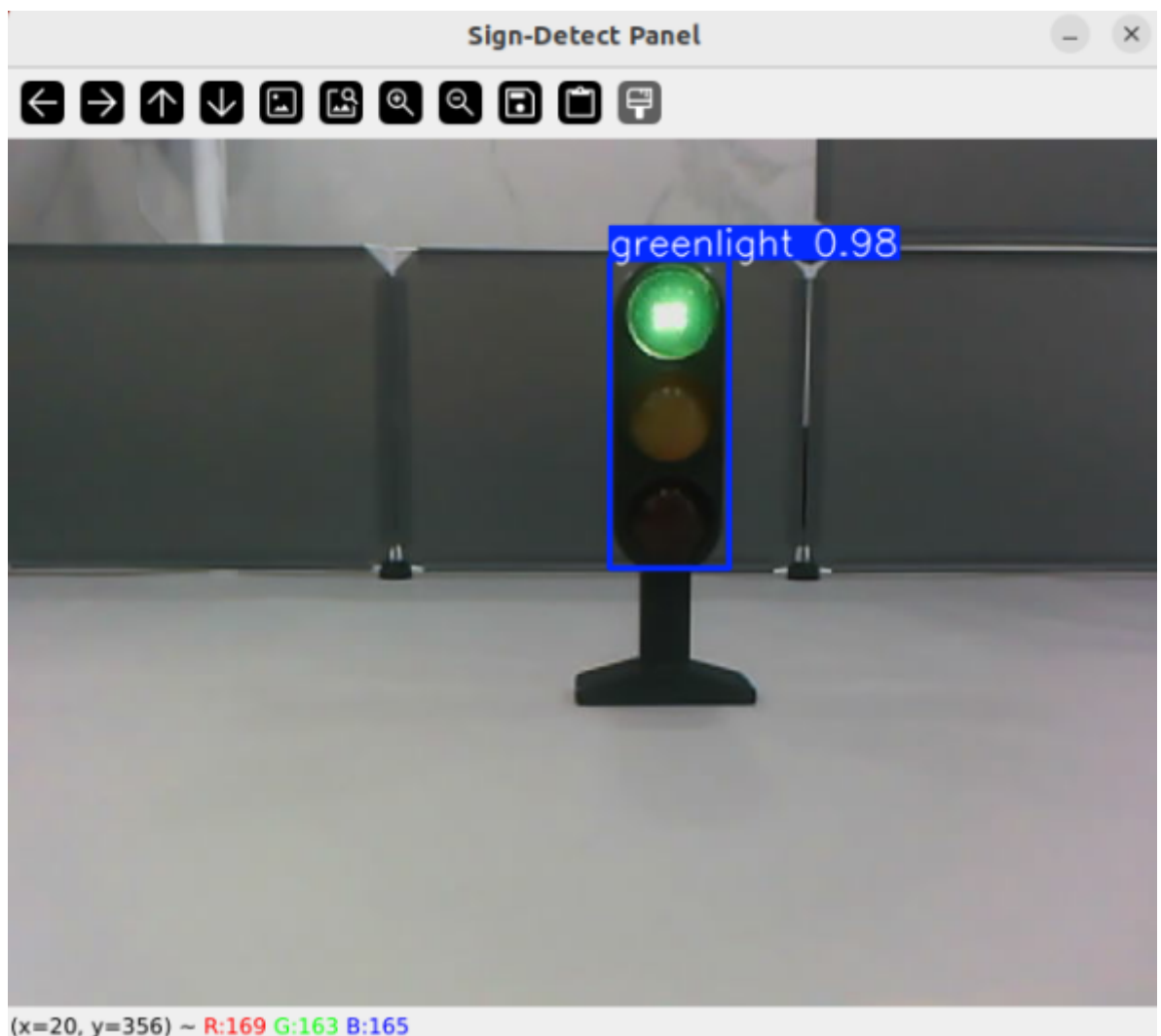
[!WARNING]

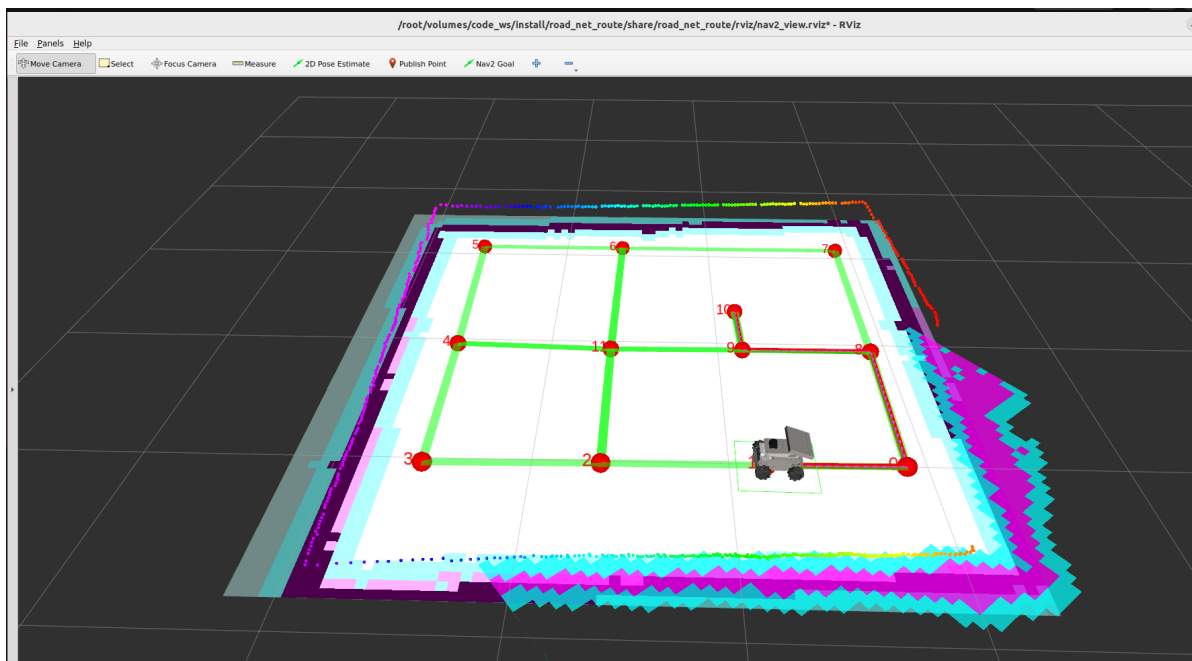
Note that the traffic light placement position should not be placed on the **path planned by the road network**, otherwise it will be detected by the lidar that there are obstacles blocking the path on the path, triggering replanning strategy





- After that, we change the traffic light to green light. When a green light is detected, it will resume road network planning autonomous driving and continue navigating to the target destination





3.4 Other Traffic Signs

- For triggering actions of other traffic signs, please refer to the instructions in [08.Autonomous Driving expand(Need purchase track map)]—[10.Traffic Sign Recognition]

4. Source Code Analysis

Source code path:

```
~/yahboomM3_ws/code_ws/src/auto_drive/auto_drive/auto_drive.py
```

- In the **init()** program, the startup parameters `combine_nav` and `unkeeplane` will be detected. If `combine_nav` is `True`, lane detection and autonomous driving will be closed; if `unkeeplane` is `True`, the lane line detection model and thread will not be loaded

```
if self.combine_nav:
    '''If combined with road network navigation, it needs to be turned
off.'''
    self.enable_auto_drive(False)
    self.recentest_pose=PoseStamped()
    self.tf_buffer = Buffer()
    self.tf_listener = TransformListener(self.tf_buffer, self)

if not self.unkeeplane:
    #Create lane line detection instances and threads
    self.lane_detect =
Lane_Detect(self.config_file,self.midline_file_path)
    if not self.lane_detect.init_yolo_engine():
        self.get_logger().error(Fore.RED+"Failed to init Lane_Detect
model."+Fore.RESET)
        raise Exception
```

```

        handle_lane_thread =
threading.Thread(target=self.handle_lane)#Create a thread to handle lane line
recognition related tasks.
        handle_lane_thread.daemon = True
        handle_lane_thread.start()

```

- The road_net_nav_sub subscriber will subscribe to the latest arbitrary pose navigation target point pose information and record it in the self.recentest_pose variable

```

#Road network navigation related
self.road_net_nav_sub= self.create_subscription(PoseStamped, "road_net_nav",
self.recentest_pose_callback, 10)
def recentest_pose_callback(self, msg:PoseStamped)->None:
    '''Record the most recent navigation destination'''
    self.recentest_pose = msg

```

- The logic for starting traffic light processing is as follows. In the combined road network navigation mode, if a red light is detected, the cancel_nav_pub topic publisher will publish a cancel navigation signal to let the road network planning node cancel navigation; when a green light is detected, if in combined road network navigation mode, the road_net_nav_pub road network publisher will republish the target pose in the most recently recorded self.recentest_pose, thereby resuming navigation to the previous target

```

def redlight(self):
    '''redlight'''
    self.get_logger().info('redlight')
    if self.combine_nav:
        self.cancel_nav_pub.publish(Bool(data=True))
    else:
        self.enable_auto_drive(False)
        self.__set_current_speed()

def greenlight(self):
    '''greenlight'''
    self.get_logger().info('greenlight')
    if self.combine_nav and self.recentest_pose is not None:
        self.road_net_nav_pub.publish(self.recentest_pose)
    else:
        self.enable_auto_drive(True)

```